Battery Managment System for 4S Li-ion Battery Pack

Author: Amer Šabić

Date: October 29, 2026

Contents

1 Introduction **2**

2 Overview of the used tool **3**
2.1 Altium Designer 3
2.2 STM32 and ESP32 4

3 System operating principle **5**
3.1 Power supply 6
3.2 Voltage measuring 6
3.2.1 Signal multiplexing 7
3.3 Current measuring and battery switch 7
3.4 Temperature measuring 7
3.5 MCUs with additional circuits 8

4 Schematics **9**

5 PCB Design **14**
5.1 Layer stack 14
5.2 Switching regulator layout 15
5.3 STM32 layout 16
5.4 ESP32 Layout 18
5.5 Isolation barrier 19

6 Software **20**
6.1 System states 20
6.2 FreeRTOS code 22
6.2.1 Acquisition task 23
6.2.2 Protection task 25
6.2.3 Communication task 27
6.2.4 Supervisor task 28

Chapter 1

Introduction

A 4S Li-ion battery pack is a lithium-ion battery configuration made of four individual cells connected in series. The nominal voltage of the package is from 14.4V to 14.8V (depending on the voltage of a single cell which varies from 3.6V to 3.7V), while the maximum value is 16.8V (fully charged).
This battery package is used for supplying many complex electronic systems such as : drones, RC cars, planes, E-bikes and so on.
Batteries in general have many different paramaters that can be of interest for measuring, since exceeding certain thresholds of voltage, current or temperature can cause damage of the supplying load, the batteries theirself or even be dangerous for the environment and people using them.

The goal of this project is to design a system that measures voltage of each cell, current drew by the load and temperature of the package in real time and transfers data wirelessly to a portable device such as PC, smartphone but also has protective role powering off the load if needed.



Figure 1.1: 4S Li-ion battery pack

Chapter 2

Overview of the used tool

This chapter covers introduction to basic hardware and software tools used in this system. Detailed explanation of other components, their roles and connecting will be presented in the next chapter.

2.1 Altium Designer

There are dozens of options when it comes to choosing the right software for PCB design and many factors by which each of the software differs from another. The program chosen for designing this system is Altium Designer because of good user interface, many additional features and for being an industrial standard in many companies.



Figure 2.1: Altium Designer

2.2 STM32 and ESP32

The essential part of any modern embedded system is a **microcontroller**. This device contains two of them (STM32 and ESP32) each one having different role. Model chosen for the STM32 is **STM32G474RE** because of many additional functionallities, faster ADCs and better efficiency (better low-power sleep and standby modes) compared to commonly used F4 series. For ESP32, **ESP32-WROOM32** is chosen even though every other ESP32 would satisfy the requirements because it is mainly being used for integrated WiFi module.

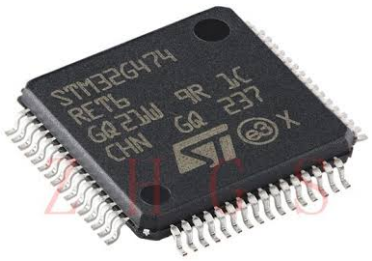


Figure 2.2: STM32



Figure 2.3: ESP32

Chapter 3

System operating principle

This chapter describes operating principle of the system, lists all the used components, explains their functionality and also shows their interconnection. There are 4 main functionalities of this device :

- Voltage measuring
- Temperature measuring
- Current measuring
- Battery disconnect switch

and each on of them will be detaily explained below. One of the most important thing about this PCB is isolation. There shouldn't be direct contact between battery side and the MCU side of the device for many reasons :

- User safety - The MCU side (ESP32, USB, WiFi, debug header) is accessible to the user. If a fault occurs on the battery side (short circuit, MOSFET breakdown), the user is safe from any harm
- Electronics protection - Overvoltage and transient that happen on the battery side cannot reach ESP32 and STM32
- The battery ground floats relative to the ground of a PC or charger. Connecting USB or a debugger to the MCU side can create ground-loop currents that inject noise or flow through measurement paths. Isolation removes this.
- Switching noise from the buck converter stays on the battery side

Because of that, several isolation chips are used in this project.

3.1 Power supply

Both of the microcontrollers require 3.3V of supply and the voltage of the battery varies from around 12V to 16.8V, which is why a buck converter is necessary. The chosen chip is LMR33630 with large input voltage range (3.8V - 36V). It contains integrated MOSFETs which is why the external ones are not needed which saves a lot of space on PCB. Maximum operating current is 3A which is more than enough for system functionality and it generates relatively small amount of EMI.

The chip used for this project requires some external components (capacitors and resistors of ceratin values) to work properly. The layout is detaily explained the datasheet.

- VCC pin is connected to GND with 1uF capacitor
- VIN pin is used for input voltage, it is also connected to 2 bypass capacitors
- EN pin is in this case connected directly to VIN, but it can also be connected to GND for disabling converter
- BOOT pin is connected to SW pin with 100nF capacitor and to an inductor which other end is the *output voltage* that is then connected to voltage divider.
- FB (feedback) pin is connected to the point between 2 resistors of the voltage divider Lastly, there is pair of 22uF output capacitors.

Also, because of 2 different isolated sides, the isolated DC-DC module was needed (NKE0305SC was chosen).

3.2 Voltage measuring

Voltage of the battery pack can vary based on the voltage of each individual cell which is why it is important to have that information in every moment. The simplest way of their precise measurment is with voltage divider. It is obvious we cannot directly connect battery with microcontroller because voltage greater than 3.3V would damage pins. The maximum operating voltage is 16.8V. For that reason alone, it would be enough to scale that voltage to around 3V or even less for safety reason (in case voltage increases rapidly), but because of maximum input voltage range of the ***isolation amplifier*** that will be shown later, the voltage to which scaling will be done is aroung 200mV.
There are 4 voltage dividers with the same values of resistor pairs : 10k and 820k.

That way, the output voltages that will be connected to MCU are : 50.6mV, 101.2mV, 151,8mV and 202.4mV representing cell voltages : 4.2V, 8.4V, 12.6V, 16.8V respectively.

3.2.1 Signal multiplexing

Connecting every voltage divider on different isolation chip would require more space and would use 4 ADCs on STM32. The alternative is to use 2 Multiplexers to which cell pins will be connected as inputs. MUX outputs are connected first to operational amplifier (*MCP6004*) for voltage buffering and then to isolation chip *AMC1301* to separate battery signal from MCU circuit.
To control MUX outputs, digital control pins are necessary and they are manipulated via STM32 pins, which means another chips are used for isolation - in this case digital *ISO7730*.

3.3 Current measuring and battery switch

The simples way of current measuring is with shunt resistor. Shunt resistor is a resistor of a small resistance which voltage is being measured across its ends and using basic Ohm's law, the current through resistor is being calculated. It is important to use resistor with small tolerance and small temperature coefficient so the resistor does not heat.

In case there is possibilty of any kind of damage, there is a security switch for switching-off the battery supply for the load. It is achieved with MOSFET that is being controlled by supplying enough voltage for the gate which will let current flow through the transistor powering the load. Since the minimum voltage needed for MOSFET to turn ON is around 10V, it was necessary to use Power MOSFET driver that will provide enough voltage from the battery source, but depending on the digital signal being driven from the MCU.

3.4 Temperature measuring

Temperature measuring is achieved with basic voltage divider with one NTC resistor which voltage is being measured for the temperature to be calculated.

3.5 MCUs with additional circuits

MCU chips cannot work with a supply voltage alone which is why the additional circuits are necessary for a microcontroller to work properly. There are several bypass capacitors to be set close to the MCU supply pins to ensure stable supply voltage. Their values are chosen based on datasheet information. The *BOOT* pin is connected to GND with a 10k resistor ensuring the chip always boots from FLASH memory. NRST pin is connected with pull-up resistor to 3.3V enabling chip reset when connected to GND. External crystal oscillator with additional capacitor is also connected to STM32 in case one is needed later. Similar circuits are present at ESP32 with 2 buttons being added for RESET and MODE change. There are also 2 headers for programming of the two microcontrollers.

Chapter 4

Schematics

This chapter shows complete electrical schematics divided into different parts based on their purpose, that were previously explained.

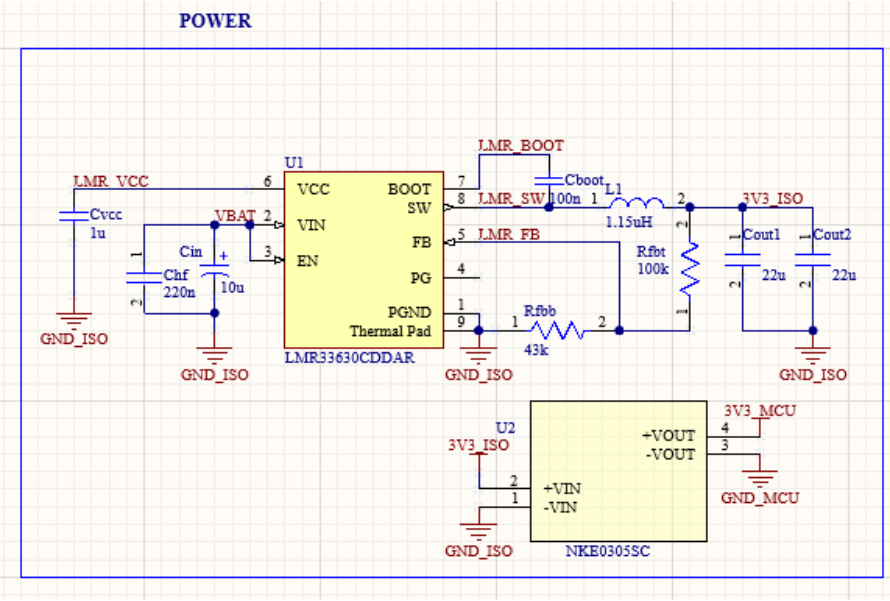


Figure 4.1: Power of the system

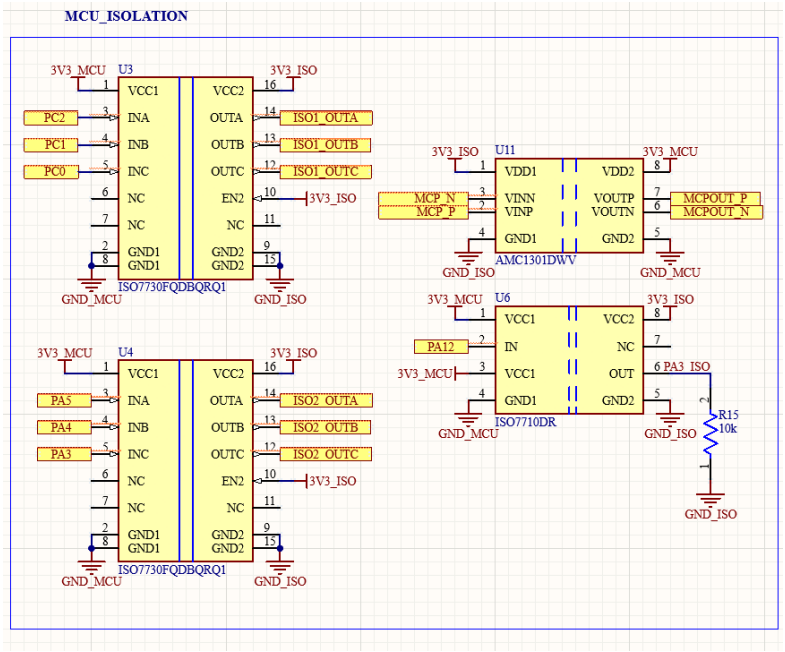


Figure 4.2: Isolation circuits

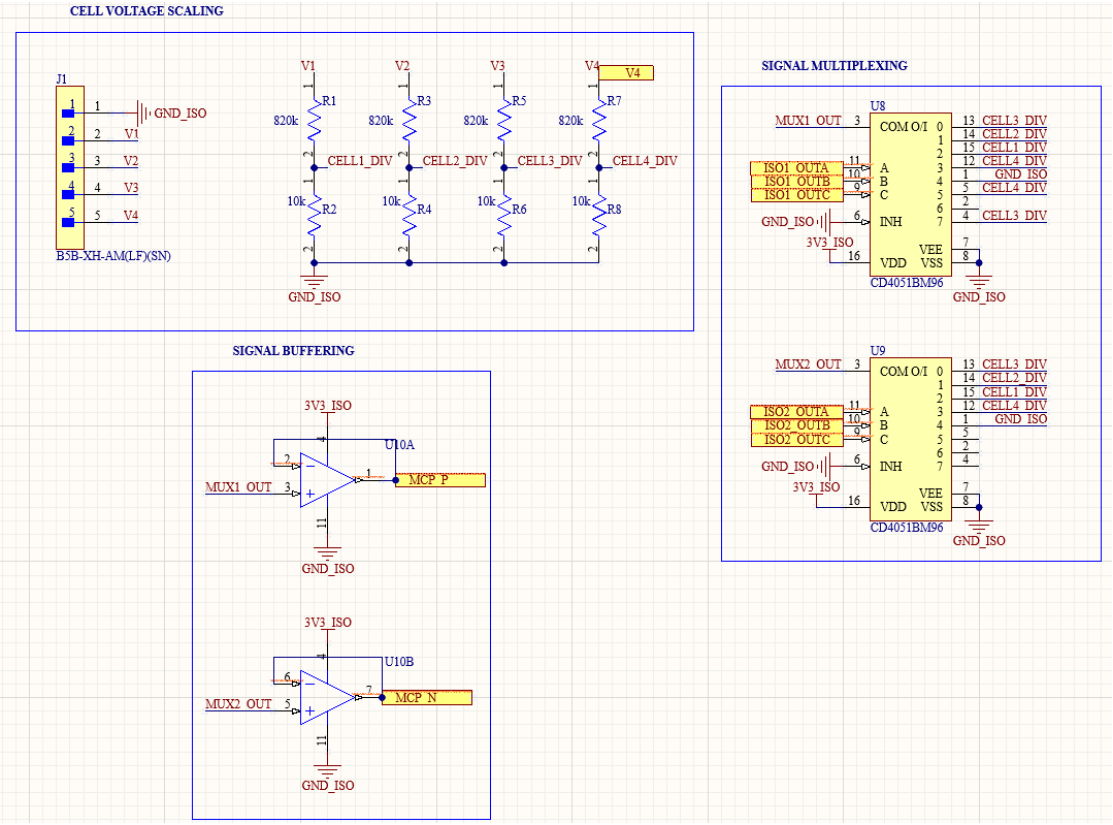


Figure 4.3: Voltage scaling, signal multiplexing and buffering

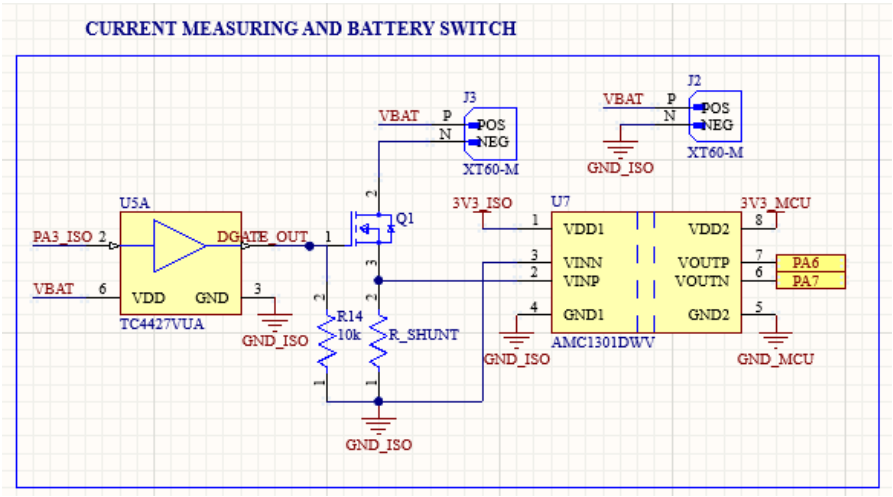


Figure 4.4: Current measuring and supply switching

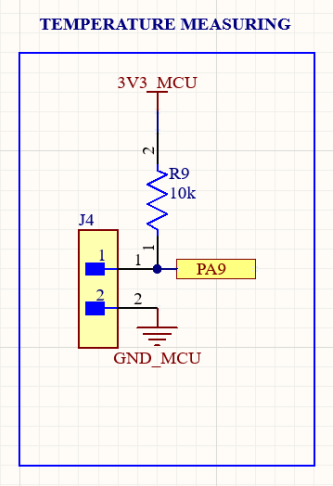


Figure 4.5: Temperature measuring

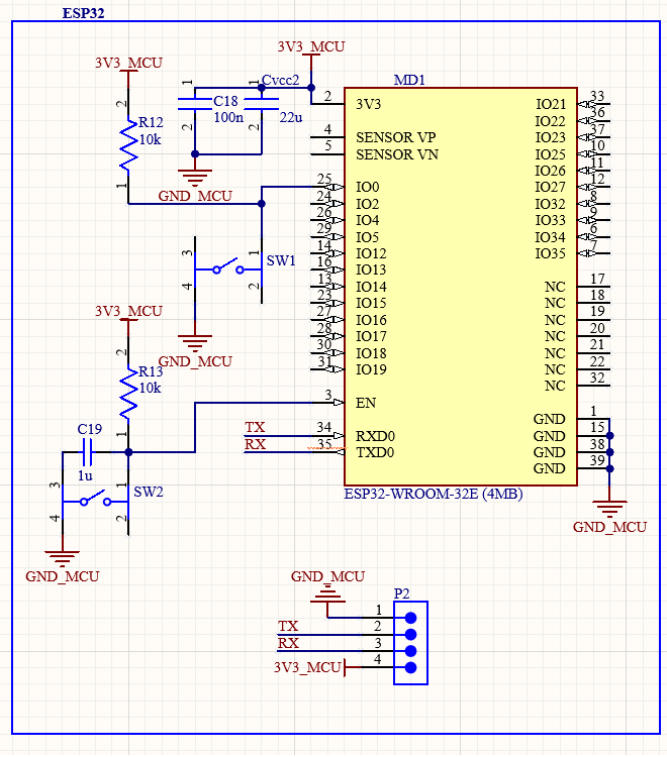


Figure 4.6: ESP32 hardware configuration

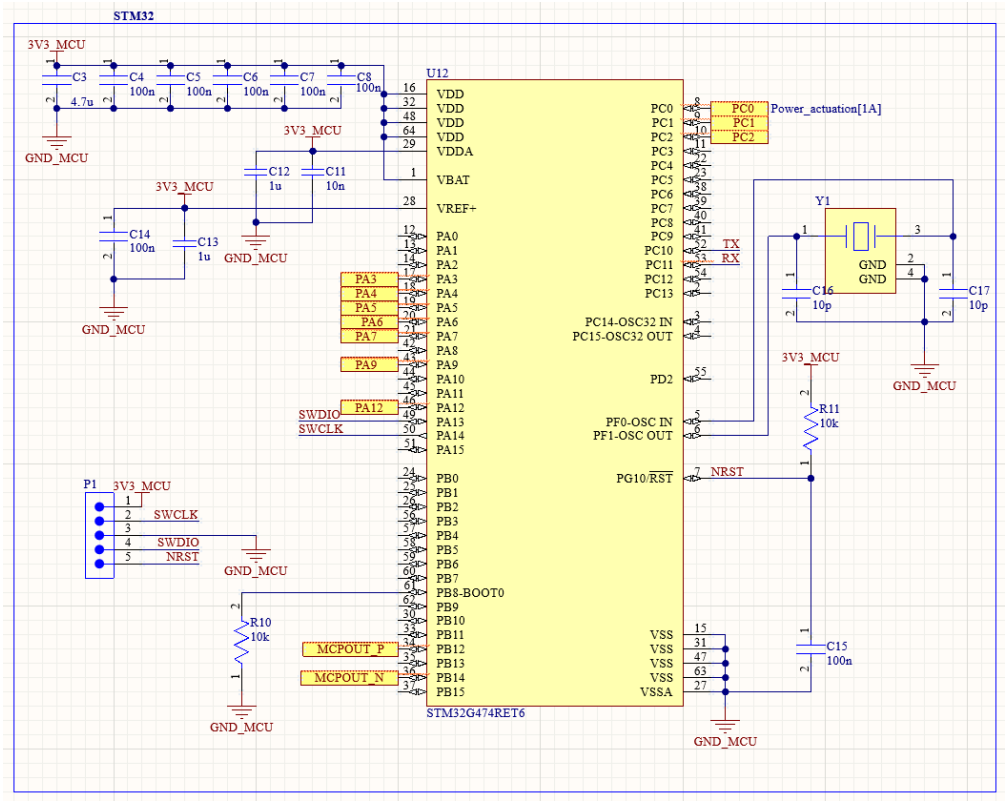


Figure 4.7: STM32 hardware configuration

Chapter 5

PCB Design

This chapter shows designing process of the 4-layer PCB following some basic rules so that each component of system works properly and ensuring the signal integrity is not compromised.

5.1 Layer stack

The board uses a four-layer stackup (Signal – GND – Power – Signal) with a total thickness of 1.6 mm, which is the standard thickness offered by most manufacturers. The outer layers use 1 oz copper for signal routing and current-carrying traces, while the inner layers use 1/2 oz copper for the ground and power planes. The outer layers are separated from the inner planes by 0.2 mm prepreg, and the two planes are separated by a 1.065 mm core.

For this board, the GND plane is placed directly beneath the top layer which gives every top-layer signal a close and continuous return path, which reduces loop area and noise coupling.

#	Name	Material	Type	Weight	Thickness	Dk	Df
	Top Overlay		Overlay				
	Top Solder	Solder Resist	Solder Mask		0.0254mm	4	
	Top Surface Fini...	PbSn	Surface Finish		0.02mm		
1	Top Layer	CF-004	Signal	1oz	0.035mm		
	Dielectric 2	PP-017	Prepreg		0.2mm	4.6	0.02
2	GND Plane	CF-004	Signal	1/2oz	0.0175mm		
	Dielectric 1	Core-039	Core		1.065mm	4.6	0.02
3	Power Plane	CF-004	Signal	1/2oz	0.0175mm		
	Dielectric 3	PP-017	Prepreg		0.2mm	4.6	0.02
4	Bottom Layer	CF-004	Signal	1oz	0.035mm		
	Bottom Surface...	PbSn	Surface Finish		0.02mm		
	Bottom Solder	Solder Resist	Solder Mask		0.0254mm	4	
	Bottom Overlay		Overlay				

Figure 5.1: The layer stack

5.2 Switching regulator layout

Unlike circuits with low signal edge rates, switching regulators (such as buck, boost, and SEPIC converters) are highly sensitive to PCB layout design. Improper component placement and trace routing can have a bad impact on efficiency, regulation accuracy, and noise immunity, or even cause high Electromagnetic Interference (EMI). This vulnerability comes mainly from high-frequency switching transients (di/dt) generated by the MOSFETs, as well as magnetic fields emitted by the inductor. To minimise loop area and noise coupling, the external components must be placed close to the chip. Given the fact that there is high-frequency switching, currents flowing through a regulator are changing very fast, inducing a voltage and because of that there are two input capacitors (10uF and 100nF) which roles are energy reservoir and decoupling. That is why they should be set at maximum of a few millimeters from the VIN pin. SW pin is the output pin of the regulator to the inductor. It connects one MOSFET to VIN and the other MOSFET to the GND (MOSFETs are integrated in the chip) changing its value from 0V to VIN a few hundreds of thousands times in a second while inductor and output capacitor are keeping the output voltage stable. This is why SW pin is the noisiest net in the regulator which is the reason why there are strict rules for layout. The voltage divider should for the same reasons be as close to the chip as possible, but at the same time relatively far away from magnetic component and SW pin. There is also a BOOT pin that supplies gate driver for high-side MOSFET. In order for it to be turned on, gate must be at a higher potential than source. Also, there are several pins connecting top layer with ground and power layers to shorten return current paths and to remove heat.

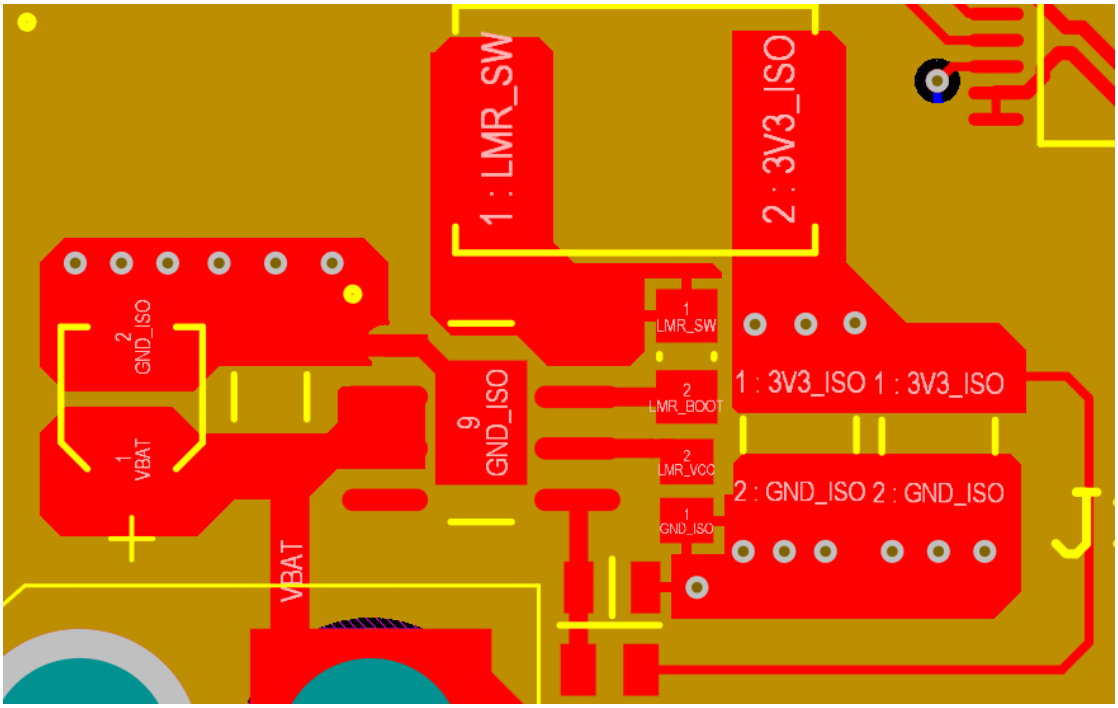


Figure 5.2: Buck regulator layout

5.3 STM32 layout

For STM32 to work properly, additional components are required just like for the switching regulator. For every VDD pin, one 100nF capacitor should be place close to the pin. Also, one larger 10uF capacitor should also be placed fairly close to the chip to serve as an energy reservoir, but it doesn't have to be as close as decoupling capacitors. There is also an external oscillator provided as an option for higher clock accuracy, but the system operates on the internal 16MHz oscillator. For an oscillator to oscillate at a certain frequency, pair of an external capacitors is required and the values are calculated with a formula :

$$C_L = \frac{C_1 \times C_2}{C_1 + C_2} + C_{stray} \tag{5.1}$$

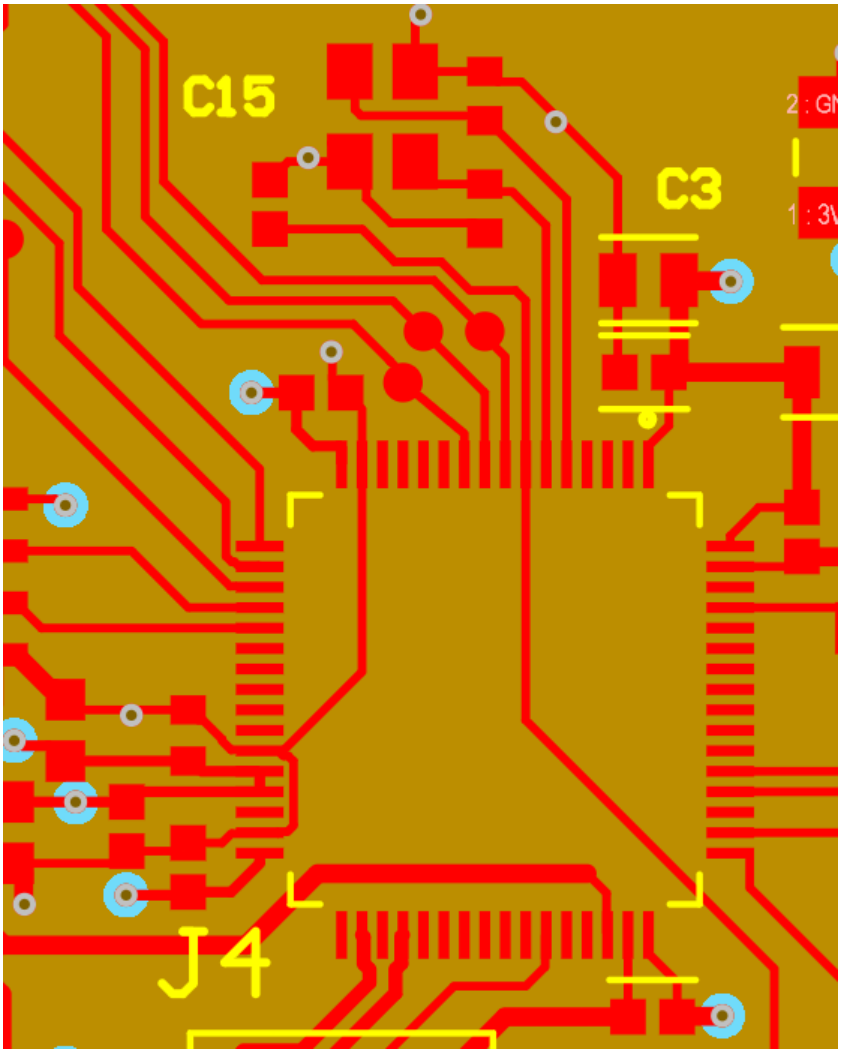


Figure 5.3: STM32 Layout

C_L is provided in a datasheet and equals 9pF, C_{stray} is approximately 4pF, so the capacitors C_1 and C_2 are each 10pF. The layout for the rest of the components (header for chip programming, reset and boot circuit) of a STM32 scheme is not critical.

5.4 ESP32 Layout

Same rules apply to ESP32 layout as for STM32 layout. Decoupling capacitors should be close to the VDD pins, the layout of other components is not strict. However, additional rule is that there must be no copper around or under the WiFi antenna because it detunes the antenna and reduces range.

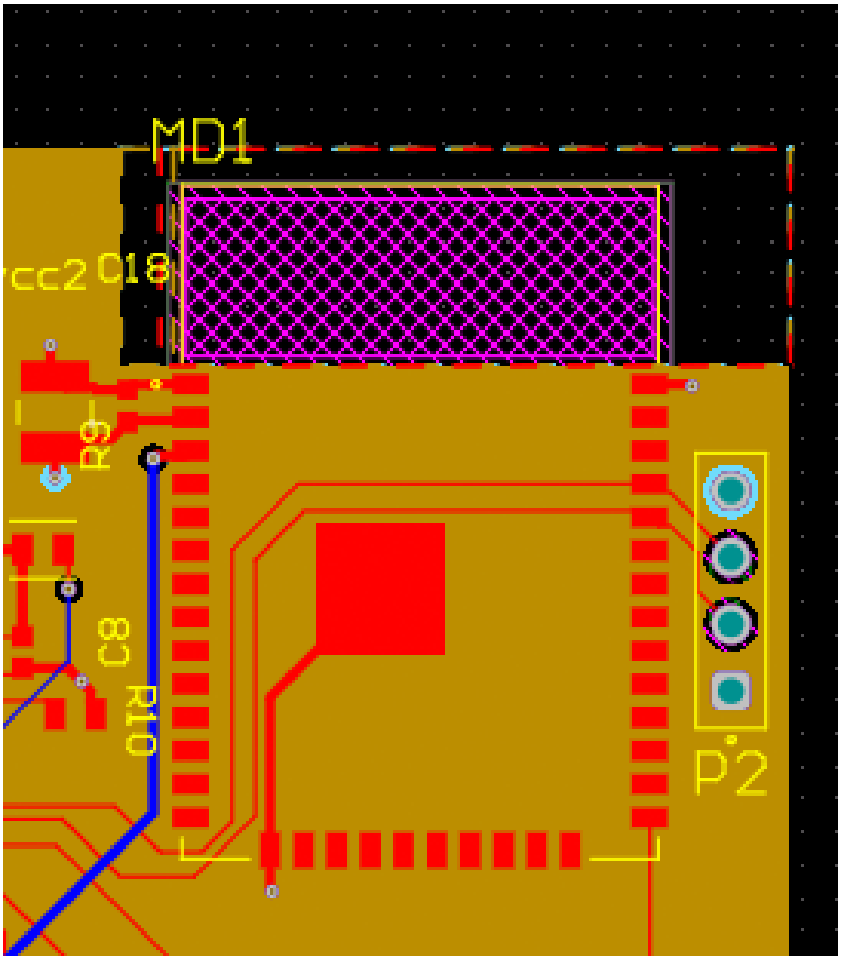


Figure 5.4: ESP32 Layout

5.5 Isolation barrier

side and those must be separated so that no trace crosses to another side directly which is the reason why isolation circuits are used. They are placed in the middle of the board and they separate two different ground and power pours as shown in the Figure 5.6.

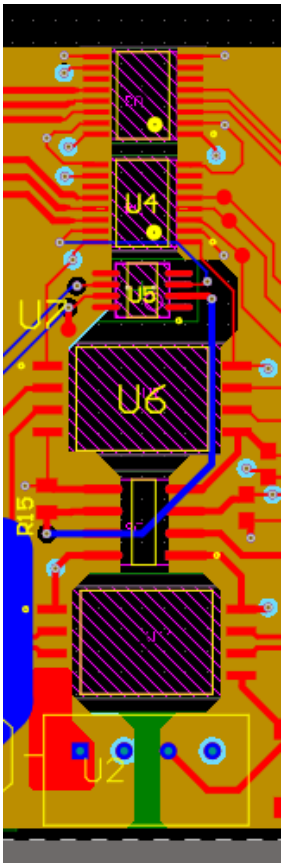


Figure 5.5: Isolation barrier

Chapter 6

Software

When developing firmware for embedded systems, there are generally two ways to structure the software. The first and simplest method is using a standard infinite loop, where commands are executed sequentially, line by line. While this approach is easy to implement, it struggles when the system needs to handle multiple time-critical operations at once. The second, more systematic approach is using a Real-Time Operating System (RTOS). Instead of a single loop, an RTOS allows the software to be divided into individual tasks, each assigned a specific priority level. The operating system then automatically manages when each task runs based on its importance. For this Battery Management System (BMS) project, FreeRTOS was utilized. Because a BMS requires constant voltage monitoring, rapid fault protection, and data transmission to happen simultaneously, designing prioritized tasks provided a much more organized, responsive, and reliable software architecture.

6.1 System states

This system is implemented as a state machine because it better represents system behaviour visually and makes code implementation much easier and more intuitive. It consists of three different states :

- **Standby state**
- **ON state**
- **Fault state**

The initial system state is standby mode. In this state load is not supplied by a battery and it will transition into an ON state only when a user enables system with a device connected to BMS via WiFi protocol while all the measurements are within chosen intervals. The system will transistion into a third *Fault state* every time at least one of the measured quantity (voltage, current, temperature) reaches upper or lower threshold, there are measurement errors or if they did not arrive within a chosen period of time. From the *fault* state, the system can only transition into a standby phase only when a user does a RESET while there are no measurement errors, overshoots or undershoots. User can also disable the device at any time chaning the state to *Standby* except from the Fault state which can transition only while previously mentioned conditions are met. The state machine diagram is shown below.

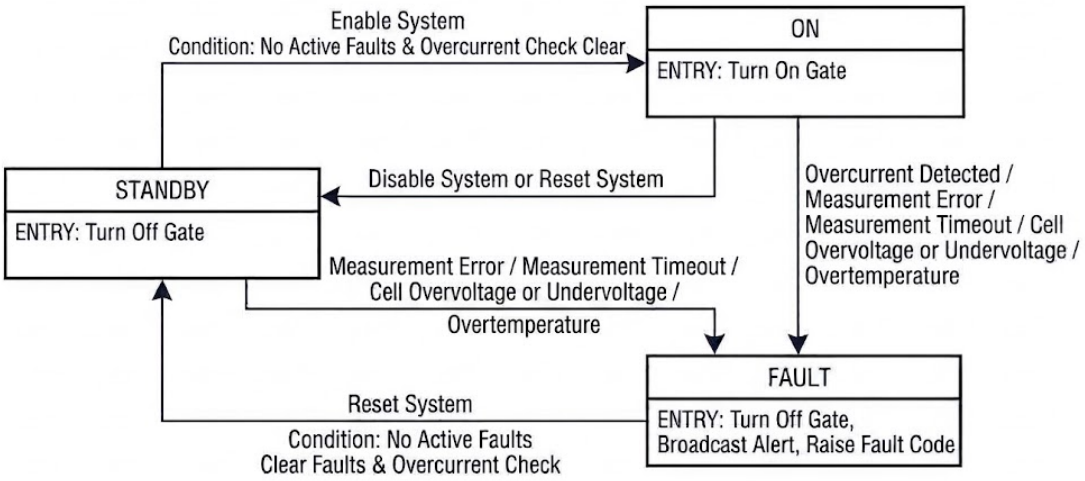


Figure 6.1: State machine diagram

6.2 FreeRTOS code

NOTE: At the time of writing this report, the physical PCB has not yet been manufactured. The assembly and soldering process will start once all ordered components arrive. Consequently, the overall system functionality cannot be fully verified at this stage. The code provided below represents the intended ESP32 firmware architecture. Following board assembly, full software debugging and hardware-in-the-loop validation will be performed. The previous section briefly explained system behaviour for certain events and conditions but this chapter will cover the FreeRTOS implementation in C programming language focusing mainly on the system tasks which there are 4:

- **Acquisition task**
- **Protection task**
- **Communication task**
- **Supervisor task**

6.2.1 Acquisition task

The acquisition task is responsible for measuring all quantities the system needs: the four cell voltages, the pack temperature, and the load current. It only measures and forwards data. It makes no decisions about thresholds, which are left entirely to the protection task.

```
void vAcquisitionTaskFunction(void *pvParameters){
    MeasData_t m = {0};
    for(;;){
        m.cell[0] = 0;m.cell[1] = 0;m.cell[2] = 0;m.cell[3]
           = 0;m.temp = 0;m.curr = 0;
        for(int i=0;i<4;i++){
            cellFunctionArr[i]();
            vTaskDelay(pdMS_TO_TICKS(2));
            HAL_ADC_Start(&hadc);

            if(HAL_ADC_PollForConversion(&hadc,10) ==
               HAL_OK){
                float cellVoltage = HAL_ADC_GetValue(&hadc)
                   / 8.2f / 4095.0f  * 83.0f * 3.3f;
                m.cell[i] = cellVoltage;
            }
            else{
                send_event(EVENT_MEAS_ERROR,0);
            }
        }
        HAL_ADC_Start(&hadc_temp);
        float temperature = 0;
        if (HAL_ADC_PollForConversion(&hadc_temp, 10) ==
           HAL_OK) {
            temperature = HAL_ADC_GetValue(&hadc_temp) /
               4095.0 * 3.3 * k;
        }
        else{
            send_event(EVENT_MEAS_ERROR,0);
        }
        m.temp = temperature;
        m.curr = current;
        xQueueSend(measQueue, &m, 0);
        xEventGroupSetBits(wdgEvents, ACQ_BIT);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

As previously mentioned, cell voltages are measured through two analog multiplexers, so only one ADC channel is needed. For each of the four cells the task performs the following steps:

- **Channel selection** - A function from the array *cellFunctionArr* sets the select pins of both multiplexers so that the chosen pair of taps is routed to the isolated amplifier.
- **Settling delay** - vTaskDelay of 2 ms lets the multiplexer and the input filter settle. Using the RTOS delay releases the CPU to other tasks.
- **Conversion** - The conversion is started on ADC4 and read by polling with a 10 ms timeout. Polling is acceptable here because the measurement is slow (a few samples per 100 ms) and the conversion itself takes only microseconds.
- **Scaling** - The raw ADC value is converted back to the real voltage.
- **Error handling** - If the conversion times out, the task does not use any value. Instead it posts an EVENT_MEAS_ERROR event to the protection task through the event queue.
- **Temperature and current measuring** - The temperature is read from a separate ADC (ADC5) connected to the NTC divider, using the same start, poll and timeout pattern. The current is not measured here. It is acquired continuously by ADC2 with DMA and monitored by the analog watchdog, so the task simply copies the most recent value from the shared variable current into the package.
- **Data transferring** - At the end of each cycle the task does two things:
  - **xQueueSend(measQueue, m, 0)** - copies the whole measurement package into the queue read by the protection task. The zero timeout means the task never blocks if the queue is full.
  - **xEventGroupSetBits(wdgEvents, ACQ_BIT)** - reports to the supervisor that the task completed a full cycle. This is the task's liveness signal and it is sent regardless of the measured values, so a faulty battery never triggers a watchdog reset.

6.2.2 Protection task

```
void vProtectionTaskFunction(void *pvParamters){
    MeasData_t m = {0};ProtEvent_t event;
    ProtState_t state = STANDBY;
    ProtStatus_t st = {0};
    for(;;){

        while(xQueueReceive(eventQueue, &event, 0) == pdPASS)
            state = handle_event(state, event, &st, m);

        if(xQueueReceive(measQueue,&m,pdMS_TO_TICKS(200)) ==
           pdTRUE){
            uint32_t f = check(&m);
            if(f && state != FAULT){
                state=FAULT;gateOFF();
                st.faultCode |= f;
            }
        }
        else if(state != FAULT){
            state = FAULT;
            gateOFF();
            st.faultCode |= FAULT_MEAS_TIMEOUT;
        }
        st.state = state;
        TxPackage_t TxP;
        TxP.m = m;
        TxP.st = st;
        xQueueSend(transmissionQueue, &TxP, 0);
        xEventGroupSetBits(wdgEvents, PROT_BIT);
    }
}
```

The protection task is the only place where decisions are made. It owns the state of the system (STANDBY, ON, FAULT), it is the only code that may switch the MOSFET gate on, and it receives all information through two queues: measurements from the acquisition task and events from everywhere else.

- **Initial state** - The task starts in STANDBY with the gate off. The gate is never switched on automatically after boot or after a watchdog reset. It has to be enabled explicitly.
- **Event handling** - At the start of each iteration the task drains eventQueue with a zero timeout, so urgent events are processed before the next measurement is evaluated. The events come from different sources: the analog watchdog interrupt (EVENT_OVERCURRENT), the acquisition task (EVENT_MEAS_ERROR), and the communication task (EVENT_ENABLE, EVENT_DISABLE, EVENT_RESET). The function handle_event implements the state transitions:

  - Fault events (overcurrent, measurement error, timeout) always switch the gate off, record the cause in faultCode and move the system to FAULT.
  - EVENT_ENABLE switches the gate on only if the system is in STANDBY, the latest measurements are within limits and no overcurrent has been flagged.
  - EVENT_DISABLE switches the gate off and returns to STANDBY.
  - EVENT_RESET is accepted only from FAULT and only if the measurements are back in range. It clears the fault code and returns to STANDBY, with the gate still off, so a second EVENT_ENABLE is required to reconnect the load.
- **Measurement check** - The task then waits up to 200 ms for a measurement packet. If one arrives, check() compares the four cell voltages, the temperature and the current with their thresholds and returns a bit mask, where each bit is one fault cause (overvoltage, undervoltage, overtemperature, overcurrent). A bit mask is used instead of a single code because several faults can be active at once and all of them should be visible. On any fault the gate is switched off first, then the causes are stored, then the state changes.
- **Timeout** - If no measurement arrives within 200 ms (twice the acquisition period), the acquisition task is assumed to have failed. Without measurements the task cannot know whether the pack is safe, so it does not assume it is: the gate is switched off and FAULT_MEAS_TIMEOUT is recorded.
- **Report** - At the end of each iteration the current measurement and the status (state and fault code) are packed into a TxPackage_t and sent to transmissionQueue with a zero timeout. Also, PROT_BIT is set in the event group as the liveness signal.

6.2.3 Communication task

```
void vEspTxTaskFunction(void *pvParameters){
    TxPackage_t TxP;
    static uint8_t frame[48];
    HAL_UARTEx_ReceiveToIdle_DMA(&huart_esp, rxBuf,
       RX_BUF_LEN);
    for(;;){
        if(ulTaskNotifyTake(pdTRUE,0) > 0){
            rx_parse(rxBuf, rxLen);
            HAL_UARTEx_ReceiveToIdle_DMA(&huart_esp, rxBuf,
               RX_BUF_LEN);
        }
        if(xQueueReceive(transmissionQueue, &TxP,0) ==
           pdTRUE){
            size_t length = build_frame(frame,&TxP);
            HAL_UART_Transmit_DMA(&huart_esp,frame,length);
        }
        xEventGroupSetBits(wdgEvents, COMM_BIT);
    }
}
```

The communication task transmits telemetry and receives commands, and it translates valid commands into events. It does not decide whether a command is executed.

- **Reception** - Reception is started once with DMA and the UART idle-line detection. DMA stores incoming bytes into rxBuf without CPU involvement. When the line goes idle, the interrupt callback stores the received length and wakes the task with a task notification. The interrupt handler is therefore very short, and the parsing is done in task context. When notified, the task calls rx_parse, which checks the start byte, the length and the CRC16 of the frame. Frames with a wrong CRC are discarded, so a corrupted byte can never become a valid command. A valid command is converted into an eventand put into eventQueue.

- **Transmission** - If a package is available in transmissionQueue, build_frame serializes it field by field (cell voltages, temperature, current, state, fault code) into the frame (start | type | length | payload | CRC16), and DMA sends it.

6.2.4 Supervisor task

```
void vSupervisorTaskFunction(void *pvParameters){
    IWDG_Conf();
    for(;;){
        EventBits_t bits = xEventGroupWaitBits(wdgEvents,
           ALL_BIT, pdTRUE,pdTRUE, SUP_PERIOD);
        if((bits & ALL_BIT) == ALL_BIT){
            HAL_IWDG_Refresh(&hiwdg);
        }
        else{
            gateOFF();
        }
    }
}
```

Supervisor task restarts the system in case one of the task stopped running. Each monitored task sets its own bit (ACQ_BIT, PROT_BIT, COMM_BIT) in the event group wdgEvents at the end of every successful cycle. The supervisor waits up to SUP_PERIOD for all bits to be set (xWaitForAllBits = pdTRUE) and clears them on exit (xClearOnExit = pdTRUE), so every period starts fresh. A reset is safe by design. The gate driver has a pull-down resistor, the gate pin is initialized low, and the protection task always starts in STANDBY, so the load stays disconnected after any reset until an explicit enable command arrives.